

Relatório 2 - Prática: Linguagem de Programação Python (I)

Gustavo Nunes da Silva

1. Introdução

O objetivo deste Card 2, com base nos dois vídeos assistidos, foi montar uma base prática de Python, saindo dos fundamentos como variáveis, tipos, operadores e estruturas de controle e chegando em Programação Funcional e Orientação a Objetos.

2. Desenvolvimento

Nesses dois vídeos eu pude reparar as formas que a informação assume dentro do programa: inteiro (int), decimal (float), texto (str) e booleano (bool). Vi que um inteiro guarda a vida de um personagem, o float guarda a altura dele, já a string guarda o nome e o booleano diz se ele está vivo ou não. O que mais me chamou atenção é que aqui eu não preciso declarar o tipo antes de criar a variável, ou seja, o próprio Python descobre isso na hora, e é isso que eles chamam de tipagem dinâmica. Isso deixa a escrita bem mais rápida, mas tem um detalhe que eu não esperava: mesmo sem eu declarar nada, o Python não deixa misturar os tipos, se eu tentar somar um número com uma string ele para e acusa o erro na hora em vez de tentar adivinhar o que eu quis dizer. No começo achei chato, depois entendi que isso é proteção, porque é melhor o erro aparecer logo do que o programa seguir rodando com um valor errado. Já os operadores são os símbolos que fazem o trabalho em cima desses dados, e são de três tipos

- **Aritméticos** (+, -, *, /, %, **) são eles que fazem as contas.
- **Relacionais** (<, >, <=, >=, ==, !=) eles comparam valores e devolvem True ou False.
- **Lógicos** (and, or, not) juntam as várias condições numa só.

Nessa parte eu tive um pouco de dificuldade porque no começo as quatro estruturas parecem a mesma coisa mas cada uma resolve um problema diferente, e o que usei para decidir foi memória e velocidade de busca, a lista é a mais solta de todas ela é mutável, aceita item repetido e cada elemento tem sua posição então eu uso ela quando o dado cresce ou muda durante a execução já a tupla é o oposto disso porque depois de criada ninguém mais consegue alterar o conteúdo dela e é justamente por isso que ela serve pra dado constante e pra retorno múltiplo de função, o set também é mutável só que ele não aceita repetição de jeito algum e é ele que eu chamo quando preciso limpar a duplicata de uma massa grande de dados já o dicionário trabalha com par de chaves e valor ele não aceita chave repetida e eu uso quando quero buscar uma propriedade direto pelo nome dela sem precisar varrer a coleção inteira indo atrás.

Passei a usar Tupla no lugar de Lista sempre que o dado não muda mais depois de criado porque com isso eu ganho duas coisas com isso ninguém altera aquilo por acidente lá na frente e ela ainda ocupa menos memória já o Set virou meu atalho favorito para limpar as duplicatas e antes eu ia escrever um laço comparando item por item agora eu só converto a lista pra set e acabou tornando o meu trabalho mais rápido e eficaz porque ele já resolve isso sozinho em tempo constante

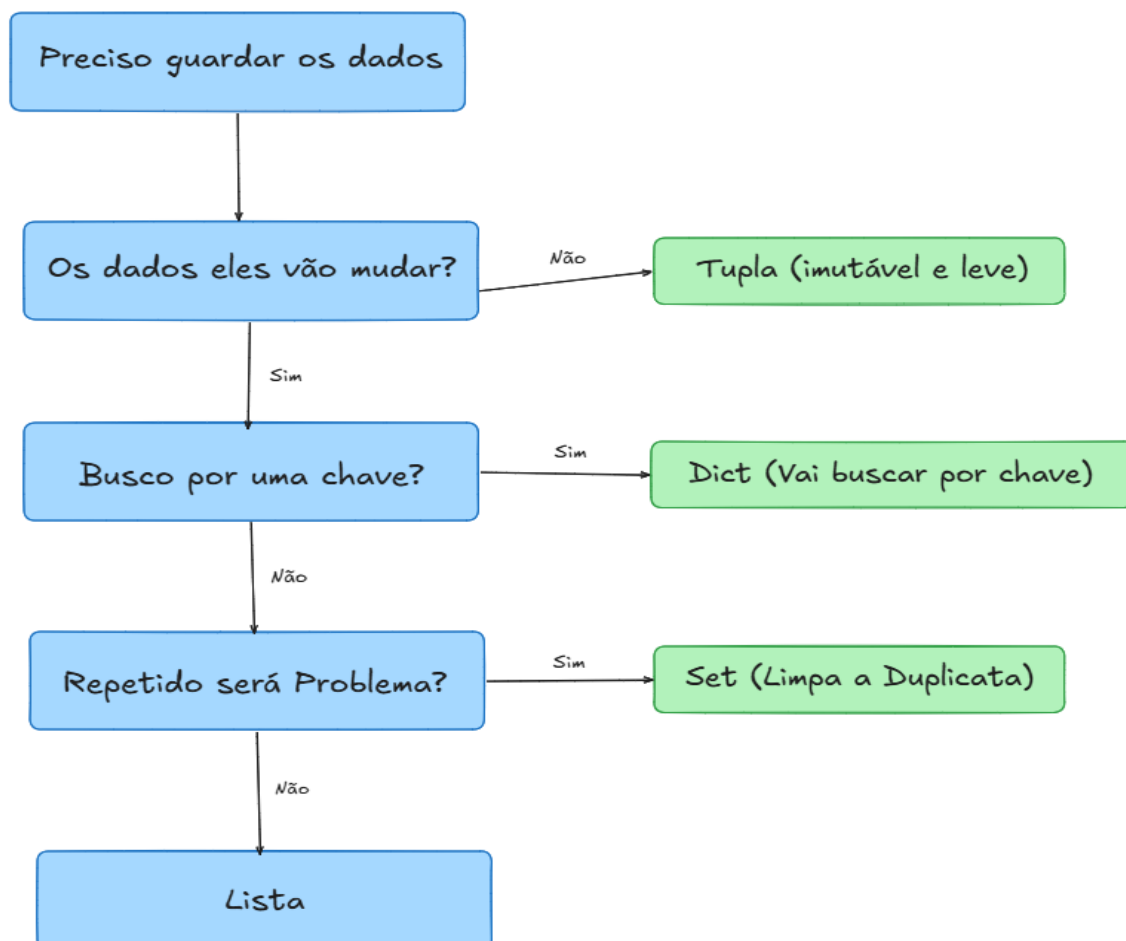


Figura 1 – Fluxograma de decisão: qual estrutura de dados usar

Fonte – Autoria própria (elaborado no Excalidraw)

Vi no vídeo também a questão das estruturas de controle e repetição e vi que o if que é o que cria a decisão e o elif é o (senão se) que por acaso podemos ter vários e o else é a saída geral que roda quando nada antes bateu ainda nas condições teve uma coisa que achei muito interessante para saber se uma lista está vazia eu não preciso escrever `if len(lista) > 0:`, dá para escrever só `if lista:`. E isso já funciona porque o Python já entende a coleção vazia como falsa por natureza o tal do truthy/falsy eu particularmente gostei desse jeito porque o código fica menor e quem faz a checagem é o próprio interpretador e é isso que a comunidade chama de Pythonic ai em seguida eu vi as estruturas de repetição que são duas e a diferença entre elas é saber ou não quantas voltas o código vai dar

- **for**: percorre uma coleção que já existe como uma (lista, tupla, dicionário). Uso é quando sei exatamente o que vou varrer.
- **while**: ele repete enquanto a condição continuar verdadeira. Serve para quando não dá para saber o número de voltas antes, tipo um menu que só para quando o usuário escolhe sair.

O que mais me pegou no while foi ter que lembrar de atualizar a variável da condição dentro do laço se eu esquecer ela nunca vai ficar falsa e o programa trava em loop infinito no for isso não acontece porque quem controla o fim é a própria coleção.

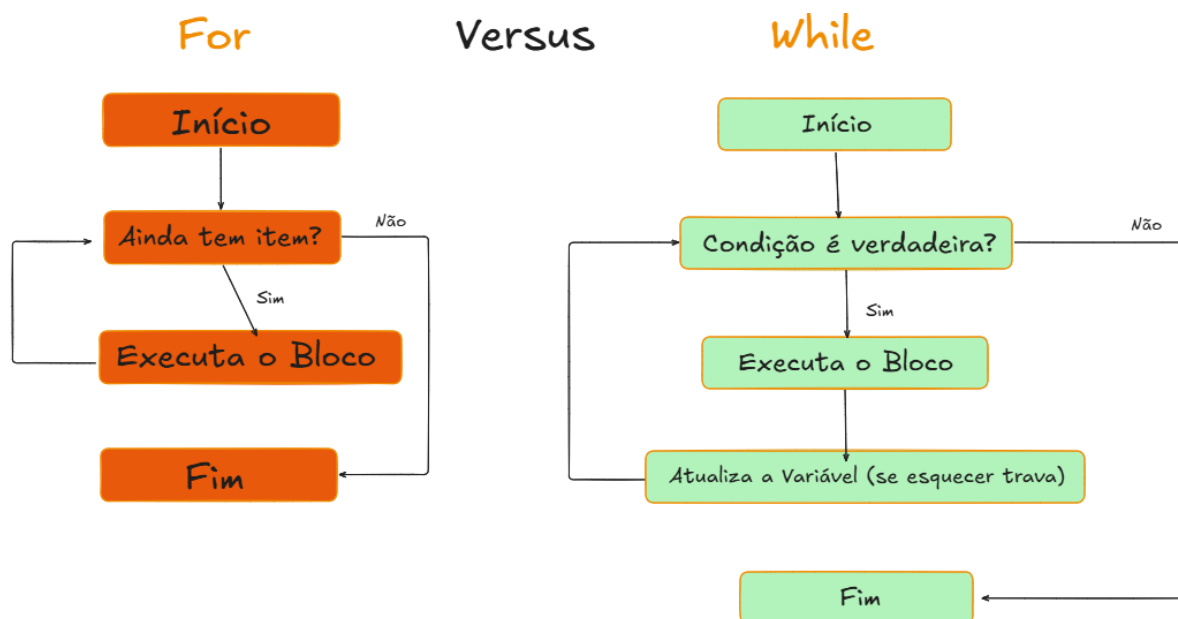


Figura 2 – Fluxograma comparando for e while

Fonte – Autoria própria (elaborado no Excalidraw)

Vi também que as funções servem para organizar o código e não deixar tudo virar um bloco gigante dentro da main podendo só mostrar algo na tela ou processar e devolver algum resultado e existem ainda as lambda que são funções pequenas escritas numa linha só e sem nome são boas pra lógica curtas usada uma vez como checar se a nota é maior ou igual a 7 achei interessante mas se a lógica crescer é melhor voltar pro def normal porque a lambda comprida fica ilegível junto com isso apareceu o empacotamento de argumentos onde o *args junta uma quantidade indefinida de argumentos numa tupla e o **kwargs tem a função de juntar os nomeados num dicionário e isso me chamou atenção porque uma função de soma que só aceita dois números quebra quando alguém manda três e já com o *args ela recebe quantos vierem, foi a partir das funções que entrou o paradigma funcional, que aparece quando a função vira um valor qualquer e pode ser guardada numa variável ou passada como parâmetro pra outra função e é o que faz o filter ele separa os itens que passam numa condição e o map transformar cada item e o reduce tem a função de juntar tudo num valor só sendo que esse último no Python 3 não vem pronto e precisa de from functools import reduce comparando as formas de transformar uma lista eu achava que map e filter eram sempre os mais rápidos que o for comum e não são como chamam a função a cada item eles perdem para List Comprehension que faz o mesmo numa linha só e roda otimizada em C, a principal vantagem deles é outra

é segurar a memória em volume grande, um resolve velocidade e o outro resolve memória.

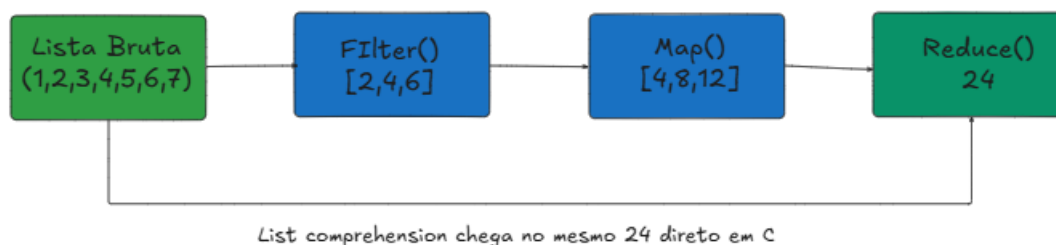


Figura 3 – Pipeline funcional com filter, map e reduce

Fonte – Autoria própria (elaborado no Excalidraw)

E por último veio a orientação a objetos onde a classe é o molde e o objeto a cópia feita a partir dele então uma classe Produto com nome, preço e desconto gera canetas e cadernos com a mesma estrutura e seus próprios valores vi no vídeo também a herança, quando a classe filha aproveita o que a pai já tem, como exemplo a Mago que herda a vida e a força de Personagem, e o super, depois veio o encapsulamento, com dois underlines na frente do atributo (preço) e o @property e o setter pra acessar de forma controlada. Aqui teve algo que só entendi depois quando comecei a fazer os testes o duplo underline não torna o atributo privado de verdade o Python só o renomeia por trás, então é mais um aviso de não mexa. O que protege de fato é o setter porque concentrando a alteração nele a regra vale sempre tipo não deixar o preço ficar negativo.

3. Conclusão

O que ficou mais claro nesses dois vídeos é que o Python não tem várias formas de fazer a mesma coisa por acaso: cada estrutura existe pra um cenário, e escolher certo separa um código que funciona de um que se sustenta. A List Comprehension ganha do laço tradicional em performance, e map e filter valem quando o problema é memória. Encapsular não é enfeite, garante que a regra de negócio seja aplicada num lugar só. No fim, levo menos sintaxe decorada e mais justificativa pra cada escolha.